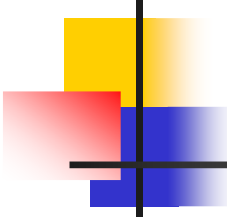


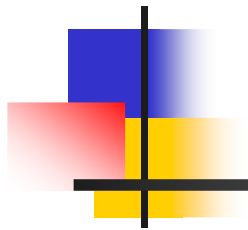
Introduction to Makefiles

ACEnet Workshop
Michelle Shaw
September 8, 2008

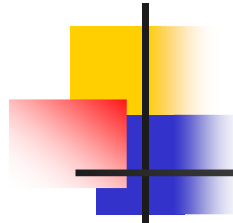


What is covered...

- The basics:
 - Why use it and what is it?
 - Overview of the parts of a Makefile
 - What make does with this file
- The parts of a Makefile:
 - Variables
 - Rules
 - dependencies
- More complex examples:
 - Multiple Makefiles
 - The include directive
 - Parallel make



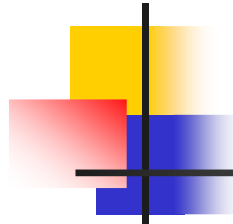
The basics...



Why use make??

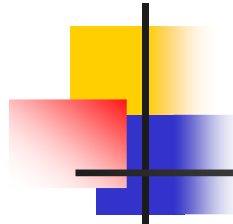
- Imagine a set of C source files – file1.c, file2.c, ..., file5.c to build program.x
 - Needs 6 steps to build:

```
pgcc -c file1.c
pgcc -c file2.c ...
pgcc -o program.x file1.o file2.o ... file5.o
```
- Each time you update the source, you need to repeat at least 2 steps!
- With make, you put this into a file and use make to execute these steps with one command



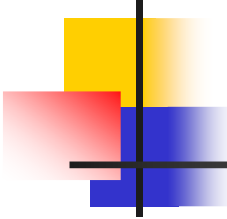
Why use make??

- For larger codes, the combination of individual compile steps is just too large
- Useful for building packages since many linux packages install via the “3-step process”
- Some don't auto-configure or this step is not done properly, so you will need to edit the Makefile yourself



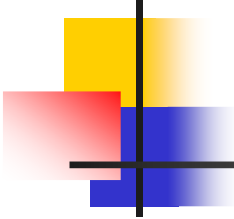
What is Make?

- Make is a package which describes how to rebuild a package
- Consists of the command *make* and a set of user-defined files called Makefiles
- Build with a single command: *make*
- Standard across many unix and linux flavors (with some exceptions)
- This talk will be based upon GNU make



What is a Makefile?

- *Makefile* – file containing all the information needed to rebuild the package
- Describes how the program files are related to one another
- set of rules which give the commands to update the program
- No two Makefiles are identical and the layout depends on the author and the program



Sample Makefile

CC = gcc

CFLAGS =

LD = gcc

LDFLAGS = -lm -lc

C_OBJS = setparams.o read_ene.o read_proj_v2.o gen_xas_cpmd.o \
rd_demon_inp.o linebroaden.o test_setparams.o

all: \$(C_OBJS)

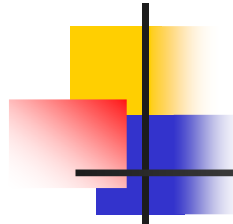
\$(LD) -o gen_xas \$(LDFLAGS) \$(C_OBJS)

.c.o:

\$(CC) \$(CFLAGS) -c \$*.c

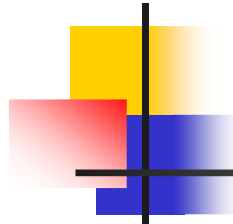
clean:

rm *.o gen_xas



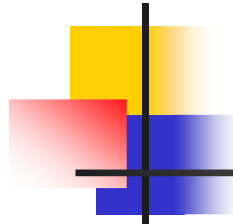
Makefiles – the pieces

- Variable declarations:
 - Define compilers, flags, linker, etc.
- Dependencies
- Directives:
 - Done while make is processing the Makefile
 - Example: the *include* statement
 - Make includes the file that comes after
 - Useful for collecting variables for use with multiple makefiles



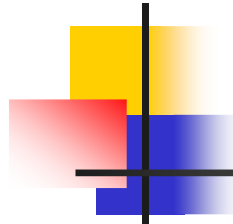
Makefiles – the pieces

- Rules for updating the object and executable files (implicit and explicit):
 - Rule for compiling one or more source file
 - How to link them together
 - How to clean up object and/or exe files
 - How to install the executables/libraries/include files
- Comments:
 - Useful for documentation
 - Begin with a #



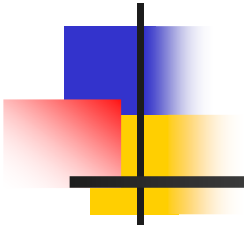
What make does

- make begins by processing the first target
- It looks for any dependencies required to complete the task for that target
- If there are rules for the dependencies, it processes them first
- If the files exist, it checks the time stamp to see if they have changed

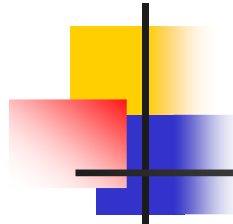


What make does

- Object/executable files that don't exist are built from scratch
- Any rules which are not listed in the default rule (usually the first one), they are not processed
- The unprocessed rules can be processed by issuing *make rule_name*

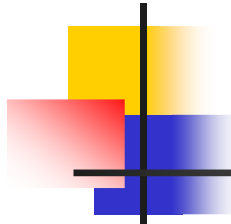


Parts of a Makefile



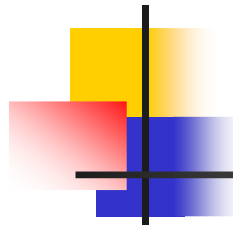
Variables

- Variable declaration is necessary to identify key components in the build process
- Some examples of things that can be declared via variables are:
 - Compilers and their respective flags
 - Binary program names and object file names
 - Architecture
 - Preprocessor and flags



Variables

- Variables are declared in a fashion similar to shell variables, with the exception that the names can contain spaces
- The general format is:
 - *VARIABLE=value*
- Examples of variable declaration:
 - Compilers: *FC=pgf90*
 - Flags: *FFLAGS=-fast*
 - Object files: *OBJS=file1.o file2.o*

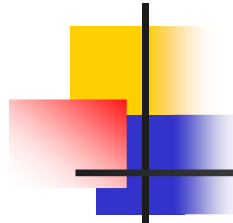


Variables

- These variables can then be accessed in rules and command lines using \$(VAR), e.g. :

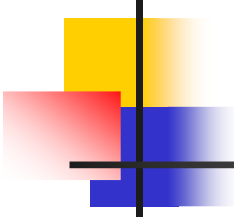
\$(FC) \$(FFLAGS) -c \$.f*

- The brackets tell make that the name includes all the characters of the name
- Without them it would interpret the name using only the first character
- Computed variables (like indices in loops) are accessed using \$\$index



Variables

- Variables are also useful for listing executable or object files for later referencing in a rule
- Using variables in Makefiles is similar (but not identical!) to shell scripting
- *A notable difference:* make takes the last defined value of a variable and keeps it for the life of the build process (unlike shell scripting)



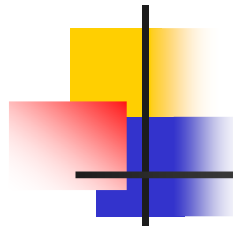
Default Variables

- some variables have default values – to see these, type the following in a directory with no makefiles:
 - *make -p*
- Some default variables and their values:
 - MAKE – “make”
 - CC – “cc”
 - AR – “ar”
 - RM – “rm -f”
 - FC – “f77”



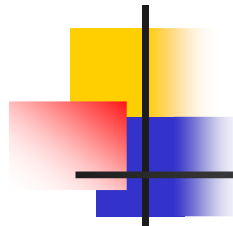
Automatic Variables

- Variables which can change as the makefile is processed are called *automatic variables*
- Convenient short-hand notation to use in rules with no specified filenames
- Some automatic variables:
 - `$@` - target name
 - `^` - dependency names without duplicates
 - `+` - dependency names with duplicates included
 - `<` - name of first dependency in the rule
 - `?` - names of dependencies that have been modified
 - `*` - useful with a suffix rule: base of the target's name



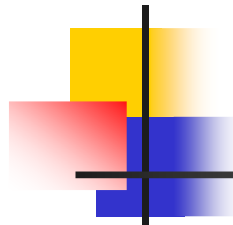
Rules

- Rules describe a set of actions to complete to achieve a goal
- Types of goals:
 - Build a set of object files
 - Build an executable
 - Set up a directory structure
 - Remove unnecessary object files



Rules

- Types of actions include:
 - Compile Fortran code
 - Preprocess C code
 - Make directories
 - Remove .o and .mod files
 - Link object files to create an executable



Rules

- General format of a rule:

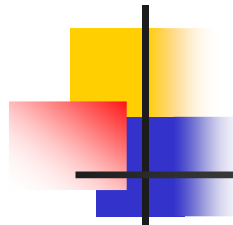
target: dependencies

command1

command2

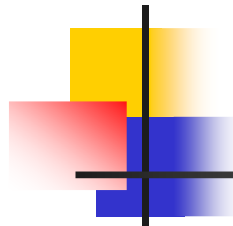
...

- *target* – name of an executable or command
- *dependency* – file or action needed before the rule's goal can be achieved
- *command1, command2, ...* - "actions" required to achieve the target



Rules

- NOTE: *the format is not very flexible*
- Commands in rules MUST be preceded by at least one tab
- Each command must either fit on one line or be separated by a \
- Makefile can contain many rules
- Rules which are not depended on by other rules must be called explicitly
 - Example: “clean” to clean out object files



Rules

- Some example rules:

all: \$(OBJ)

\$(FC) \$(LDFLAGS) -o test_incgma \$(OBJ)

.f.o:

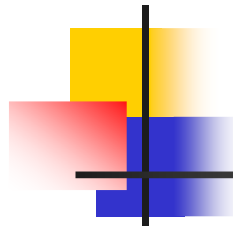
\$(FC) \$(FFLAGS) -c \$.f*

clean:

*rm *.o test_incgma*

package: \$(PACK_OBJS) \$(PACK_LIBS)

*\$(FC) \$(LDFLAGS) -o package \
\$(PACK_OBJS)\$(PACK_LIBS)*



Rules

- Some rules don't contain any commands
- Called *pseudo (or phony) targets*
- Useful for listing targets, e.g. building more than one program, each with a separate rule
- Example:

all: exe1 exe2 ...

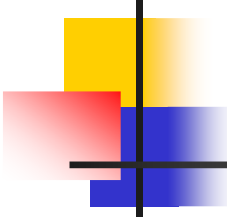
exe1: \$(OBJ1)

\$(CC_BUILD) -o \$@ \$(OBJ1)

exe2: \$(OBJ2)

\$(CC_BUILD) -o \$@ \$(OBJ2)

...

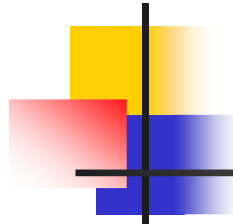


Implicit Rules

- Rules like "exe1: \$(OBJ1)"... are *explicit rules*
- Other useful rules don't explicitly define the files to build
- These are called *implicit rules*
- Example – combine rules for all .o files with a .c extension:

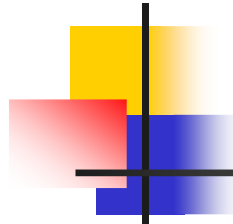
.c.o:

\$(CC_BUILD) -c \$<



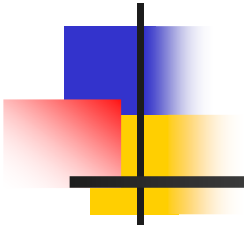
Dependencies

- Package executables depend on object files being built
- these dependencies are usually listed in a variable like this:
VAR=obj1.o obj2.o ...
- Dependency list referenced using \$(VAR)
- They can be grouped according to the type of source they are built from, e.g. Fortran or C

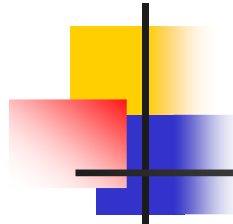


Dependencies

- *A note of caution:* list of dependencies may have a specific order if one object file in the list depends on something in another object file in the list
- The order that the object files are listed is important in this case

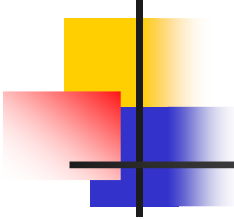


More complex Makefiles



Multiple Makefiles

- In some cases, Makefiles are spread across several source directories
- In this case, variables common to all Makefiles can be put in a global include file
- Include it using:
include \$TOPDIR/make.inc
- Local variables can then be put directly into each individual Makefile



Includes in Makefiles

- Using the previous example:
 - The make.inc file:

```
CC      = gcc
CFLAGS  =
LD       = gcc
LDFLAGS = -lm -lc
```



Includes in Makefiles

- The rest of the Makefile:

```
include ./make.inc
```

```
C_OBJS = setparams.o read_ene.o read_proj_v2.o\  
        gen_xas_cpmd.o rd_demon_inp.o \  
        linebroaden.o test_setparams.o
```

```
all: $(C_OBJS)
```

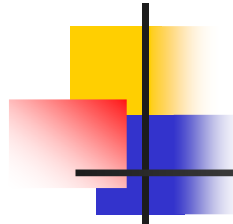
```
    $(LD) -o gen_xas $(LDFLAGS) $(C_OBJS)
```

```
.C.o:
```

```
    $(CC) $(CFLAGS) -c $*.c
```

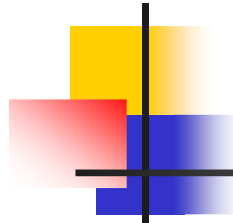
```
clean:
```

```
    rm *.o gen_xas
```

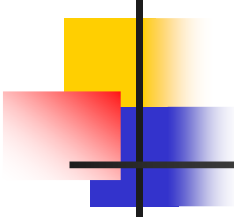
Includes in Makefiles

- The include statement is very useful when multiple Makefiles use the same set of variables
- Simplifies greatly the editing of such files
- A real-world example: Quantum-ESPRESSO



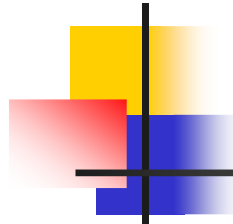
A more complex example

- One can also use Makefiles to build separate packages and coordinate their builds with a high-level Makefile
- Used in codes like Quantum-ESPRESSO and Elmer-FEM
- Especially handy when the parts of the package to build are independent of one another
- Example: set of source files to build/update as a set and grouped into directories by analysis type



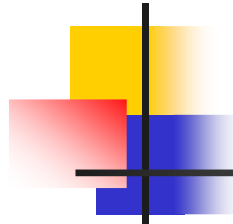
A more complex example

- In root dir:
 - Makefile to control the calls to the Makefiles in the subdirectories
 - make.inc to control settable variables
 - Five directories, each with their own Makefiles
 - To see options, type “make”
 - Choose from the list of options which codes to build



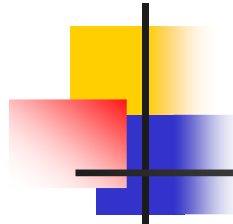
A more complex example

- README in the example `make_ex2.tar.gz` explains the directory layout and options to make
- Instructions to build the example is found with the source files in `README_bldex`
- Build the example, then play around with the options in `make.inc`
- Any suggestions to improve this setup?



Parallel make

- make can be run in parallel
- significantly speeds up the build process
- Instead of just "make", use:
make -j <number of processors>
- Note that the order that the source files are compiled is not defined
- can have nasty side effects if the object files have some order in which they need to be built



Parallel make

- Run the second example in parallel by changing the variable MAKE from:

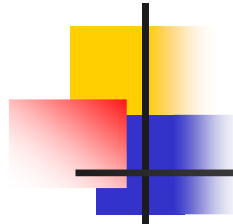
MAKE = make

to, say:

MAKE = make -j 2

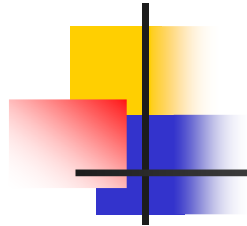
then accessing this variable with *\$(MAKE)*

- This will run make with 2 processors



Some final notes

- Make can do much more but for most of what you will do this is sufficient
- Building Makefiles is part of the process of automating your tasks
- Best way to learn how to write Makefiles is by experience!



More Information

- GNU make manual
- Internet in general
- ACEnet support: support@ace-net.ca
- Your local CRC